

摘要：

首个龙虾AI基准测试排行榜出炉，32款模型同台竞技，Gemini 3 Flash以95.1%成功率夺冠，国产MiniMax与Kimi双双杀入全球前三，完胜GPT-4o。扎心的是，Claude旗舰版成本高达竞品近200倍，准确率反不及中端模型。

你现在养了几只龙虾？

这就是现在打招呼最常见的问题，上周腾讯深圳总部排起龙等着免费装 OpenClaw，真是一代人有一代人的鸡蛋。



连黄仁勋也盛赞 OpenClaw 为「有史以来最重要的软件发布」，认为它已经证明了 AI 在高度个性化环境中，能够完美复刻人类的复杂 workflow。

养龙虾太过火爆，也终于出现专门针对 OpenClaw 的基准测试 PinchBench，用于评估大语言模型在 OpenClaw 任务中的表现。

[README](#) [MIT license](#)



# PinchBench

## Real-world benchmarks for AI coding agents

[leaderboard](#) [pinchbench.com](#) [license](#) [MIT](#)

PinchBench measures how well LLM models perform as the brain of an [OpenClaw](#) agent. Instead of synthetic tests, we throw real tasks at agents: scheduling meetings, writing code, triaging email, researching topics, and managing files.

Results are collected on a public leaderboard at [pinchbench.com](#).



### Why PinchBench?

Most LLM benchmarks test isolated capabilities. PinchBench tests what actually matters for coding agents:

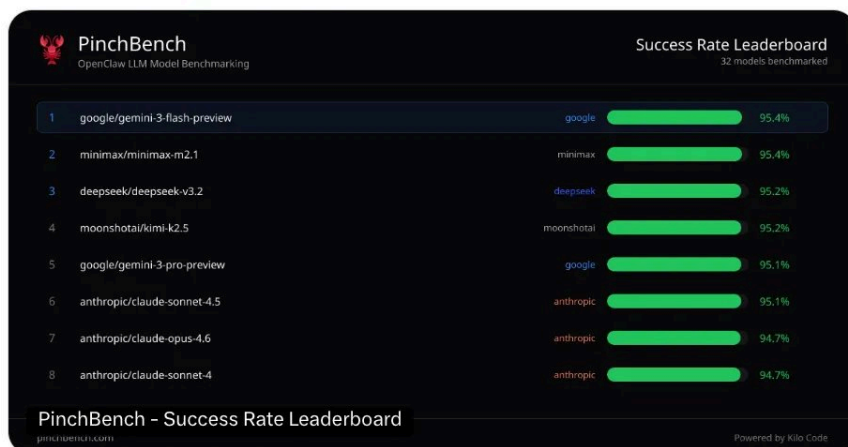
- Tool usage — Can the model call the right tools with the right parameters?
- Multi-step reasoning — Can it chain together actions to complete complex tasks?
- Real-world messiness — Can it handle ambiguous instructions and incomplete information?
- Practical outcomes — Did it actually create the file, send the email, or schedule the meeting?

PinchBench 评分方式也很硬核，有的任务看代码能不能跑通（自动化检查），有的看写得好不好（Claude Opus 当评委），还有的是两者结合。所有题目和答案都开源在 GitHub 上，谁都可以去验货。



Interesting benchmark on which model is best for [@openclaw](#)

关于哪个模型最适合 [@openclaw](#) 的有趣基准测试



From [pinchbench.com](#)

今天，OpenClaw 创始人 Peter Steinberger 分享这个龙虾基准测试排行榜。

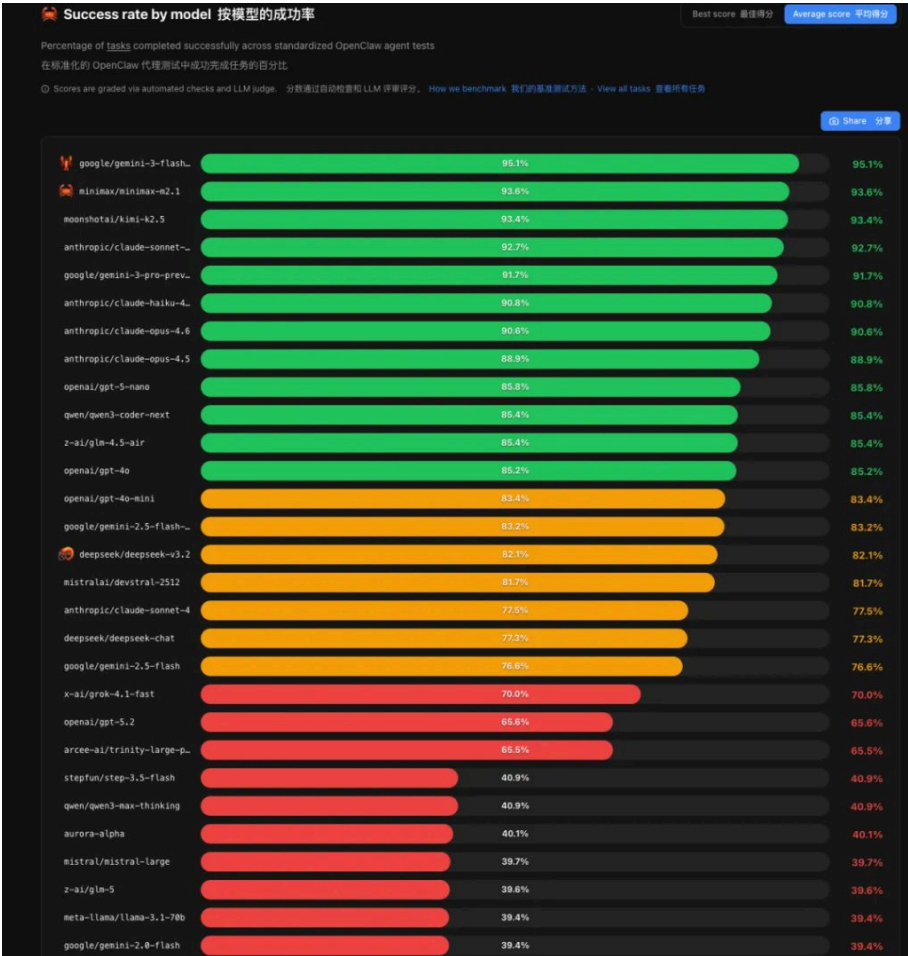


PinchBench 一口气测了 32 款主流大模型，从成功率、速度、费用三个维度，看看哪个模型最适合养龙虾。

PinchBench 官网：<https://pinchbench.com/>

## Gemini 3 Flash 成功率最高，国产模型也杀疯了

来看最重磅的成功率排名。



Google 的 Gemini 3 Flash Preview 以 95.1% 的成功率夺冠，这个成绩说实话让我有点意外。因为 Flash 系列一直是 Gemini 的「轻量版」，主打快和便宜，没想到这次在准确率上直接把自家 Pro 老大哥和 Claude、GPT 系列全超了。

这说明 Google 在模型效率优化上是真的下了功夫。轻量模型不代表能力弱，关键看怎么调。

第二名是 MiniMax M2.1，成功率 93.6%。国产模型真的站起来了，MiniMax 的表现相当亮眼，成功压过了 Claude Sonnet 4.5（92.7%）和 GPT-4o（85.2%）。

Kimi K2.5 紧随其后，成功率 93.4%。Kimi 的长文本能力一直很强，这次在编程任务上也证明了自己。和 MiniMax 一起，国产双雄直接占据了 TOP3 的两个席位。

再往后看，Claude Sonnet 4.5 排第四（92.7%），Gemini 3 Pro 第五（91.7%），Claude Haiku 4.5 第六（90.8%）。

有意思的是，Claude Opus 4.6 作为 Anthropic 的旗舰大模型，成功率只有 90.6%，排在第七。

看来「大」不一定「强」，至少在编程这个场景下，中端模型反而更香。



## 唯快不破，MiniMax 赢麻了

在开发这些重度任务中，谁都不想对着屏幕干等。速度接影响干活的心情。

MiniMax M2.5 以 105.96 秒的成绩拿下速度冠军，完成全部测试任务。什么概念？比第二名 Gemini 2.0 Flash 只快了 0.09 秒，但第一就是第一。

 Speed by model

Fastest submission time per model (Best run)

| RANK | MODEL                         | PROVIDER   | BEST TIME ⓘ |
|------|-------------------------------|------------|-------------|
| 1    | minimax/minimax-m2.5          | minimax    | 105.96s     |
| 2    | google/gemini-2.0-flash       | google     | 106.05s     |
| 3    | meta-llama/llama-3.1-70b      | meta-llama | 106.14s     |
| 4    | google/gemini-1.5-pro         | google     | 106.85s     |
| 5    | mistral/mistral-large         | mistral    | 107.72s     |
| 6    | qwen/qwen3-max-thinking       | qwen       | 109.06s     |
| 7    | z-ai/glm-5                    | z-ai       | 109.27s     |
| 8    | sourceful/riverflow-v2-pro    | sourceful  | 109.85s     |
| 9    | aurora-alpha                  | openrouter | 120.12s     |
| 10   | anthropic/claude-sonnet-4     | anthropic  | 137.66s     |
| 11   | stepfun/step-3.5-flash        | stepfun    | 142.08s     |
| 12   | google/gemini-2.5-flash       | google     | 167.79s     |
| 13   | google/gemini-2.5-flash-lite  | google     | 189.48s     |
| 14   | openai/gpt-4o                 | openai     | 190.20s     |
| 15   | mistralai/devstral-2512       | mistralai  | 195.01s     |
| 16   | openai/gpt-5-nano             | openai     | 202.12s     |
| 17   | anthropic/claude-haiku-4.5    | anthropic  | 215.06s     |
| 18   | openai/gpt-4o-mini            | openai     | 227.19s     |
| 19   | qwen/qwen3-coder-next         | qwen       | 234.66s     |
| 20   | x-ai/grok-4.1-fast            | x-ai       | 238.34s     |
| 21   | google/gemini-3-pro-preview   | google     | 239.55s     |
| 22   | minimax/minimax-m2.1          | minimax    | 239.79s     |
| 23   | openai/gpt-5.2                | openai     | 246.98s     |
| 24   | deepseek/deepseek-chat        | deepseek   | 249.47s     |
| 25   | google/gemini-3-flash-preview | google     | 254.50s     |
| 26   | anthropic/claude-opus-4.5     | anthropic  | 263.88s     |

第三名 Llama 3.1 70B（106.14 秒）、第四名 Gemini 1.5 Pro（106.85 秒）、第五名 Mistral Large（107.72 秒）——这几个差距都不大，基本在同一梯队。

但往下看就有意思了。

Claude Sonnet 4 用了 137.66 秒，比第一梯队慢了 30 秒。Gemini 3 Pro 更是用了 239.55 秒，是 MiniMax M2.5 的两倍多。

这说明一个规律：**轻量级模型普遍更快**。如果你做的是快速原型开发、需要频繁迭代，选轻量模型准没错。但如果是那种「跑一遍就行」的任务，等等大模型也无妨。

## 怎么养龙虾最划算

养龙虾，精打细算很重要，毕竟很多OpenClaw 任务都是 Token 无底洞，稍不留神就能让你怀疑人生。



Cost by model

Show \$0 Results

Lowest submission cost per model (best run)

| RANK | MODEL                         | PROVIDER   | BEST COST |
|------|-------------------------------|------------|-----------|
| 1    | openai/gpt-5-nano             | openai     | \$0.03    |
| 2    | google/gemini-2.5-flash-lite  | google     | \$0.05    |
| 3    | mistralai/devstral-2512       | mistralai  | \$0.10    |
| 4    | openai/gpt-4o-mini            | openai     | \$0.13    |
| 5    | minimax/minimax-m2.1          | minimax    | \$0.14    |
| 6    | z-ai/glm-4.5-air              | z-ai       | \$0.16    |
| 7    | google/gemini-2.5-flash       | google     | \$0.20    |
| 8    | moonshotai/kimi-k2.5          | moonshotai | \$0.20    |
| 9    | x-ai/grok-4.1-fast            | x-ai       | \$0.24    |
| 10   | qwen/qwen3-coder-next         | qwen       | \$0.38    |
| 11   | deepseek/deepseek-chat        | deepseek   | \$0.45    |
| 12   | anthropic/claude-haiku-4.5    | anthropic  | \$0.64    |
| 13   | google/gemini-3-flash-preview | google     | \$0.72    |
| 14   | deepseek/deepseek-v3.2        | deepseek   | \$0.73    |
| 15   | openai/gpt-5.2                | openai     | \$1.09    |
| 16   | google/gemini-3-pro-preview   | google     | \$1.48    |
| 17   | openai/gpt-4o                 | openai     | \$2.08    |
| 18   | anthropic/claude-sonnet-4.5   | anthropic  | \$3.07    |
| 19   | anthropic/claude-opus-4.5     | anthropic  | \$5.52    |
| 20   | anthropic/claude-opus-4.6     | anthropic  | \$5.89    |

GPT-5 Nano 以 0.03 美元的成本成为全场最便宜的选择，成功率 85.8%。虽然准确率不算顶尖，但这个价格.....还要什么自行车？适合预算有限、对错误容忍度高的场景。

Gemini 2.5 Flash Lite 排第二，只要 0.05 美元，成功率 83.2%。这个性价比就很不打了——成本是 GPT-5 Nano 的不到两倍，成功率只低了 2.6 个百分点。

MiniMax M2.1 排第五，成本 0.14 美元，但别忘了它的成功率是 93.6%。算下来每百分点的成本只有 0.0015 美元，性价比极高。

再看高端模型的成本，就有点触目惊心了。

Claude Opus 4.6 完成测试要花 5.89 美元，是 GPT-5 Nano 的将近 200 倍。但它的成功率只有 90.6%，比 MiniMax M2.1 还低了 3 个百分点。

这账怎么算都不划算。除非你对 Claude 有特殊的品牌信仰，否则从纯性价比角度，中端模型显然是更理性的选择。

## 龙虾养殖怎么选

看完三个维度的榜单，相信你已经有了自己的判断。这里 APPSO 再给大家几个场景化的建议：

如果你追求成功率，无脑选 Gemini 3 Flash

95.1% 的成功率 + 0.72 美元的成本，目前综合表现最优。适合对代码质量要求高的生产环境，出错的代价远大于模型成本的时候，选它准没错。

⚡ 如果你追求速度，选 MiniMax M2.5 或 Gemini 2.0 Flash

都在 106 秒左右完成全部任务，适合快速原型开发、需要频繁迭代的场景。时间就是金钱，这俩能帮你省下不少耐心。



如果你追求性价比，选 Gemini 2.5 Flash Lite

0.05 美元的成本，83.2% 的成功率，是入门「养龙虾」的最佳选择。个人项目、小团队、预算有限的场景，闭眼入。

如果想少折腾倾向国产模型，MiniMax M2.1 和 Kimi K2.5 都很能打

MiniMax M2.1 成功率 93.6% 排第二，Kimi K2.5 成功率 93.4% 排第三，两款国产模型都已经跻身第一梯队。而且 MiniMax 的速度还是冠军，性价比也极高，值得重点关注。

从这次 PinchBench 的榜单可以看出，Agent 已经进入了「百花齐放」的时代。Google 的 Gemini 系列在效率和成本上全面领先，国产模型 MiniMax 和 Kimi 紧随其后，OpenAI 和 Anthropic 则在高端市场保持竞争力。

对于开发者来说，好消息是选择越来越多了。坏消息是……选择困难症可能更严重了。

但没关系，记住一个原则：**没有最好的模型，只有最适合你场景的模型**。生产环境看成功率，原型开发看速度，个人项目看性价比，按需选择就好。

而且，APPSO 也想特别提醒大家，**安装 OpenClaw 或许不用花费什么成本，但「养龙虾」消耗的 Token ，可比过去我们和 AI 对话要多得多。**

前几天 OpenClaw 在纽约办的聚会上不少用户分享自己的龙虾养殖心得，有人每个月在 Token 上的花费高达1000-2000美元，更有一位「土豪」玩家每天烧掉 10 亿 tokens，没有信仰可禁不起这么烧钱。







尝鲜 OpenClaw 可以，但它其实并不适合所有人，目前有很多任务用龙虾也并非最优解，更大的意义是可以感受 AI 带来全新的交互体验。

本文来源：[APPSO](#)

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

限时 0 元领

\* 同一用户免费领取一次

扫码 免费领取



限免

1012 收藏

分享到: 微信 微博

写评论

请发表您的评论

表情 图片

发布评论



3 条评论



MobileUser8707

VIP会员

7小时前 中国陕西省

对已有记忆的压缩提取淡化消除，需要算法实时动态的加权计算，那时候就能比较准确的进行agent的代理了。

👍 1    ↩ 回复



清晨的粥聚在一起才是开始

VIP会员

8小时前 中国安徽省

云里雾里

👍 0    ↩ 回复



愤怒的小熊

8小时前 中国四川省

这个是智能体吗？

👍 0    ↩ 回复

